MixedComplementarityProblems.jl: A Fast, Batched, Open-Source Interior-Point Solver for Mixed Complementarity Problems

David Fridovich-Keil
Department of Aerospace Engineering and Engineering Mechanics
The University of Texas at Austin
Austin, TX, USA
dfk@utexas.edu

***Abstract*—Mixed complementarity problems (MCPs) arise as the first-order optimality conditions of nonlinear programs and noncooperative games, and give a natural formulation for the multi-agent trajectory optimization problems that appear throughout robotics. The dominant solver for problems of this form is `PATH`, a mature but closed-source package. We present `MixedComplementarityProblems.jl`, an open-source, pure Julia implementation of an interior-point method for parametric MCPs that (i) matches `PATH`'s reliability on standard benchmarks and (ii) natively supports batched solving, in which many parameter instances are solved simultaneously and in parallel, either across CPU threads or on an NVIDIA GPU, behind a single solver implementation. On a multi-agent lane-change trajectory game representative of robotics planning problems, our CPU-multithreaded batched solver clears a batch of parametric instances 60–80$\times$ faster than sequential calls to `PATH`, and our GPU backend clears the same batches roughly $20\times$ faster than `PATH` [TODO: finalize the GPU-vs-CPU comparison for the trajectory game once the latest benchmark numbers are in—earlier runs showed GPU trailing CPU here, but that may no longer hold]. We describe the solver's interior-point formulation, the abstraction that lets a single solver implementation run unmodified across dense, batched-sparse, and single-large linear-algebra backends, and report benchmarks against `PATH` on both randomly generated quadratic programs and trajectory games.**

I. Introduction

Multi-agent trajectory planning problems in robotics, ranging from autonomous driving to human-robot interaction, are naturally posed as noncooperative dynamic games, in which each agent optimizes its own trajectory subject to the others' choices. The first-order necessary conditions of optimality for each agent in the game take the form of a mixed complementarity problem (MCP). Solving MCPs rapidly, therefore, becomes a core computational burden for real-time decision-making in multi-agent robotics applications.

The dominant solver for this class of problems is `PATH` [1]. `PATH` is mature, reliable, and widely used; but, it is closed-source, solves one problem instance at a time, and was not designed to support automatic differentiation of solutions with respect to problem parameters. All three pose serious, practical limitations for use in robotics: (i) developers must have access to solver internals in order to customize to the problem at hand, (ii) scenario- and contingency-based planning (e.g., [2], [3]) requires solving a *batch* of identically-structured but differently *parameterized* problems, and (iii) modern machine learning pipelines often embed optimization problem solvers as "layers" [4], and end-to-end differentiability requires us to be able to differentiate a solver's output with respect to its input parameters.

We present `MixedComplementarityProblems.jl`, an open-source, pure Julia solver for parametric MCPs that addresses all of these practical challenges. Concretely, we contribute:

- A simple and easily-customized interior-point method for parametric MCPs that matches `PATH`'s reliability on standard benchmarks, while supporting automatic differentiation of solutions with respect to problem parameters.
- A native *batched* solve mode, in which many parameter instances sharing one MCP structure are solved simultaneously, parallelized across CPU threads or an NVIDIA GPU behind a single solver implementation.
- An abstraction boundary defined by a small verb set that separates the interior-point loop and its linear algebra backend, which lets a single solver implementation run unmodified across dense, batched-sparse, and single-large-instance regimes, so that supporting a new device or linear-algebra strategy never requires touching the solver loop itself.

As Figure 1 previews and Section V quantifies, on a multi-agent lane-change trajectory game representative of robotics planning problems, clearing a batch of parametric instances with our batched solver takes a small fraction of the wall-clock time required to clear the same batch with sequential calls to `PATH`. **[TODO: replace with the top-line numerical speedup results when they are finalized]**

II. Background: Mixed Complementarity Problems

A. Problem definition

A mixed complementarity problem is specified by a map $F : \mathbb{R}^n \to \mathbb{R}^n$ and bounds $\ell, u \in (\mathbb{R} \cup \{\pm\infty\})^n$; a solution is a vector $z \in \mathbb{R}^n$ satisfying, for every i,

$$\begin{aligned} \ell_i < z_i < u_i &\implies F_i(z) = 0, \\ z_i = \ell_i &\implies F_i(z) \geq 0, \\ z_i = u_i &\implies F_i(z) \leq 0. \end{aligned} \tag{1}$$

[TODO: add reference] surveys this general form and its applications. Two special cases of (1) recur throughout this report: an unconstrained block ($\ell_i = -\infty$, $u_i = \infty$), where (1) reduces to $F_i(z) = 0$; and a nonnegativity block ($\ell_i = 0$,

[TODO: replace with rendered lane-change trajectory game schematic, showing the two vehicles' solved trajectories overlaid with a faint fan of trajectories from other initial conditions in the same batch]

(a) One multi-agent trajectory game is one instance out of a batch of parametrically related MCPs, solved simultaneously.

[TODO: replace with wall-clock-vs-batch-size plot: `PATH` (serial), CPU-batched, and GPU-batched, log-scaled y-axis]

(b) Clearing that batch: sequential `PATH` calls versus our batched CPU and GPU solves.

Fig. 1. **[TODO: finalize caption once the real figure is in]** Solving a batch of parametric trajectory-game instances (a) with `MixedComplementarityProblems.jl`'s batched interior-point solver is substantially faster than solving the same batch with sequential calls to `PATH` (b).

$u_i = \infty$), where it reduces to the complementarity condition $0 \le z_i \perp F_i(z) \ge 0$.

B. MCPs from KKT conditions

Consider a parametric nonlinear program

$$\min_x \; f(x;\theta) \quad \text{s.t.} \quad h(x;\theta) \ge 0, \tag{2}$$

with primal variables $x \in \mathbb{R}^{n_x}$, parameters θ, and Lagrange multipliers $y \in \mathbb{R}^{n_y}$ associated with h. The KKT conditions of (2) are

$$\begin{aligned} G(x,y;\theta) &:= \nabla_x f(x;\theta) - \nabla_x h(x;\theta)^\top y = 0, \\ 0 \le y \perp H(x,y;\theta) &:= h(x;\theta) \ge 0, \end{aligned} \tag{3}$$

which is exactly an instance of (1) with $z = (x, y)$: an unconstrained block G paired with x, and a nonnegativity block H paired with y. We refer to (3) as the *primal-dual* form; it is the only form `MixedComplementarityProblems.jl` exposes, since every MCP we construct in this report arises from an optimization problem or a game rather than from a raw box constraint.

C. MCPs from noncooperative games

The reduction in Section II-B extends from one decision-maker to many. Consider N agents, where agent i solves

$$\min_{x_i} \; f_i(x_i, x_{-i};\theta) \quad \text{s.t.} \quad h_i(x_i, x_{-i};\theta) \ge 0,\; c(x;\theta) \ge 0, \tag{4}$$

subject to its own constraints h_i and to constraints c shared across agents (e.g., pairwise collision avoidance), where x_{-i} denotes the other agents' decisions and $x = (x_1, \ldots, x_N)$. Writing out each agent's KKT conditions as in (3) and stacking them — with the shared constraints c coupling agents through a shared multiplier — yields a single primal-dual MCP whose solution is a generalized Nash equilibrium: a strategy profile from which no agent can unilaterally improve its own objective. **[TODO: add reference]** treats this class of problems, generalized Nash equilibrium problems (GNEPs), in general; [2], [3] apply this same MCP reduction to multi-agent trajectory games in robotics.

The lane-change trajectory game of Figure 1(a) is a concrete instance of this family: two vehicles, each minimizing a cost trading off lane-keeping, speed tracking, and control effort, coupled by a pairwise collision-avoidance constraint. We use this game as a running example for the remainder of this report. A *batch*, in the sense of Section IV, is a set of instances of this same game that share structure but differ in parameters θ — e.g., the vehicles' initial positions, velocities, or lane preferences.

D. The parametric view

Sections II-B and II-C both produce a *family* of MCPs indexed by parameters θ rather than a single problem instance, which we write $\mathrm{MCP}(\theta)$, with solution map $z^*(\theta) = (x^*(\theta), y^*(\theta))$. Treating θ as first-class, rather than building a solver around one fixed problem instance, is what licenses the two capabilities central to this report: solving $z^*(\theta)$ for many values of θ at once, in parallel (Section IV), and differentiating the solution map itself, $\nabla_\theta z^*(\theta)$, so that an MCP solve can act as a layer inside a larger differentiable pipeline [4].

III. Solver Design

TODO

IV. Batched and GPU-Parallel Solving

TODO

V. Experiments

TODO

VI. Discussion and Future Work

TODO

References

[1] S. P. Dirkse and M. C. Ferris, "The path solver: a nommonotone stabilization scheme for mixed complementarity problems," *Optimization methods and software*, vol. 5, no. 2, pp. 123–156, 1995.

[2] J. Li, C. Y. Chiu, L. Peters, F. Palafox, M. Karabag, J. Alonso-Mora, S. Sojoudi, C. Tomlin, and D. Fridovich-Keil, "Scenario-game admm: A parallelized scenario-based solver for stochastic noncooperative games," in *62nd IEEE Conference on Decision and Control, CDC 2023*. IEEE, 2023, pp. 8093–8099.

[3] L. Peters, A. Bajcsy, C.-Y. Chiu, D. Fridovich-Keil, F. Laine, L. Ferranti, and J. Alonso-Mora, "Contingency games for multi-agent interaction," *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2208–2215, 2024.

[4] B. Amos and J. Z. Kolter, "Optnet: differentiable optimization as a layer in neural networks," in *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, 2017, pp. 136–145.